Министерство науки и высшего образования Российской Федерации
Калужский филиал федерального государственного автономного образовательного учреждения высшего образования
«Московский государственный технический университет имени Н.Э. Баумана
(национальный исследовательский университет)»
(КФ МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «ИНФОРМАТИКА И УПРАВЛЕНИЕ» (ИУК)

КАФЕДРА «СИСТЕМЫ АВТОМАТИЧЕСКОГО УПРАВЛЕНИЯ» (ИУК3)

РАСЧЕТНО-ПОЯСНИТЕЛЬНАЯ ЗАПИСКА

К КУРСОВОМУ ПРОЕКТУ НА ТЕМУ:

Разработка подсистемы построения математической модели САУ в пространстве состояний по структурной схеме

Студент группы ИУК3-71Б — А.А. Исаев

(подпись, дата) (И.О. Фамилия)

Руководитель курсового проекта — А.М. Макаренков

(подпись, дата) (И.О. Фамилия)

Калуга, 2026

Министерство науки и высшего образования Российской Федерации
Калужский филиал федерального государственного автономного образовательного учреждения высшего образования
«Московский государственный технический университет имени Н.Э. Баумана (национальный исследовательский университет)»
(КФ МГТУ им. Н.Э. Баумана)

УТВЕРЖДАЮ
Заведующий кафедрой ИУК-3
_____ (Д.В. Мельников)
«___» _____ 2026 г.

З А Д А Н И Е
на выполнение курсового проекта

по дисциплине _____
Студент группы ИУК3-71Б Исаев Артём Алексеевич
Тема курсового проекта: **Разработка подсистемы построения математической модели САУ в пространстве состояний по структурной схеме**
Направленность КП: **учебный**
Источник тематики: **кафедра ИУК-3**

Задание
– Описать модели типовых звеньев в пространстве состояний и способы перехода от структурной схемы к модели всей системы.
– Разработать алгоритм построения матриц A, B, C, D по структурной схеме, включая схемы с подсистемами и петлями без инерционных звеньев.
– Реализовать подсистему в серверной части системы моделирования и вывести полученную модель в веб-интерфейсе.
– Проверить подсистему на схемах с известным ответом, оценить точность и время построения модели.

Оформление курсового проекта
Расчетно-пояснительная записка на _____ листах формата А4.
Перечень графического материала КП (плакаты, схемы, чертежи и т.п.):
– схема алгоритма построения модели – 1 лист формата А3;
– примеры построения модели и результаты проверки – 1 лист формата А3.

Дата выдачи задания «___» _____ 2026 г.

Руководитель _____ А.М. Макаренков
(подпись, дата) (И.О. Фамилия)

Студент _____ А.А. Исаев
(подпись, дата) (И.О. Фамилия)

Министерство науки и высшего образования Российской Федерации
Калужский филиал федерального государственного автономного образовательного учреждения высшего образования
«Московский государственный технический университет имени Н.Э. Баумана (национальный исследовательский университет)»
(КФ МГТУ им. Н.Э. Баумана)

КАЛЕНДАРНЫЙ ПЛАН
на выполнение курсового проекта

по дисциплине _____

Студент группы ИУК3-71Б Исаев Артём Алексеевич

Тема курсового проекта: **Разработка подсистемы построения математической модели САУ в пространстве состояний по структурной схеме**

№	Наименование этапов	Срок, план	Срок, факт	Отметка руководителя	Куратор
1	Задание на выполнение КП	3-я нед.			
2	Изучение моделей звеньев и способов построения модели схемы	6-я нед.			
3	Разработка алгоритма и реализация подсистемы	10-я нед.			
4	Проверка на схемах с известным ответом, замеры времени	13-я нед.			
5	Оформление расчетно-пояснительной записки и графической части	15-я нед.			
6	Защита	17-я нед.			

Студент _____
(подпись, дата)

Руководитель _____
(подпись, дата)

Содержание

Перечень сокращений и обозначений.....6
Введение.....7
Постановка задачи.....8
1 Модель системы в пространстве состояний.....9
1.1 Что такое модель в пространстве состояний.....9
1.2 Модели типовых звеньев.....10
1.3 Как получить модель всей схемы.....11
2 Алгоритм построения модели.....13
2.1 Общий порядок работы.....13
2.2 Проверка схемы.....13
2.3 Раскрытие подсистем.....13
2.4 Объединение уравнений звеньев.....14
2.5 Таблицы соединений.....14
2.6 Итоговые матрицы и петли без инерционных звеньев.....15
3 Примеры построения модели.....17
3.1 Последовательное соединение.....17
3.2 Параллельное соединение.....18
3.3 Замкнутый контур с неустойчивым объектом.....19
3.4 Тот же контур внутри подсистемы.....20
3.5 Контур третьего порядка.....21
3.6 Петля без инерционных звеньев.....22
4 Программная реализация.....24
4.1 Место подсистемы в системе моделирования.....24
4.2 Входные данные.....24
4.3 Основные структуры данных.....25
4.4 Функции построения модели.....25
4.5 Что получает пользователь.....26
5 Проверка подсистемы.....27
5.1 Схемы с известным ответом.....27
5.2 Надёжность решения петель без инерционных звеньев.....28
5.3 Схемы с подсистемами.....29
5.4 Время построения модели.....30
6 Руководство пользователя.....32
6.1 Построение модели схемы.....32

6.2 Работа с подсистемами.....32
6.3 Если модель не построена.....33
Заключение.....34
Список литературы.....35
Приложение А. Листинги программы.....36

Перечень сокращений и обозначений

САУ – система автоматического управления

ПФ – передаточная функция

x, u, y – векторы состояния, входов и выходов

A, B, C, D – матрицы модели $\dot{x} = Ax + Bu,\ y = Cx + Du$

M, N – таблицы (матрицы) соединений схемы

κ – число обусловленности петли без инерционных звеньев

API – программный интерфейс сервера

JSON – текстовый формат, в котором схема передаётся на сервер

Введение

Систему автоматического управления (САУ) обычно изображают структурной схемой: каждое звено — прямоугольник со своей передаточной функцией, стрелки показывают, куда идёт сигнал. Схема удобна человеку, но рассчитать по ней поведение системы напрямую нельзя: для расчёта нужны уравнения.

Программы моделирования решают эту задачу по-разному. Самый простой способ — считать блоки по очереди на каждом шаге времени: каждый блок по своему входу вычисляет свой выход и свою производную. Так работает, например, обычное моделирование в Simulink. Общих уравнений системы при этом не возникает, поэтому, чтобы узнать полюса, передаточную функцию или частотные характеристики всей схемы, требуются отдельные инструменты.

Другой способ — один раз превратить всю схему в одну модель вида «вход – состояние – выход» с матрицами A, B, C, D. По такой модели сразу получаются и переходные процессы, и полюса, и передаточная функция, и частотные характеристики. Но построить её по произвольной схеме — с несколькими контурами, подсистемами внутри подсистем и петлями, в которых сигнал проходит мгновенно, — непросто.

Курсовой проект выполнен как часть выпускной квалификационной работы «Веб-ориентированная система визуального моделирования систем автоматического управления». В проекте разработана подсистема, которая по структурной схеме, собранной пользователем в браузере, автоматически строит матрицы A, B, C, D всей системы и показывает пользователю, какой блок схемы отвечает за каждую переменную модели. Все остальные расчёты системы моделирования выполняются по этой модели.

Постановка задачи

В курсовом проекте требуется:

1. Описать, как типовые звенья записываются в пространстве состояний, и сравнить способы перехода от структурной схемы к модели всей системы.

2. Разработать алгоритм, который по схеме любой сложности строит матрицы A, B, C, D: схема может содержать обратные связи, подсистемы и петли без инерционных звеньев.

3. Предусмотреть понятные сообщения для схем, модель которых построить нельзя или ненадёжно.

4. Реализовать подсистему в серверной части (Python) и показать полученную модель во вкладке «Матрицы» веб-интерфейса.

5. Разобрать работу алгоритма на примерах и проверить его на схемах с известным ответом, оценить точность и время построения модели.

1 Модель системы в пространстве состояний

1.1 Что такое модель в пространстве состояний

Любое линейное звено можно представить как устройство со входом u и выходом y (рисунок 1). Если звено инерционное, у него есть «память»: его выход зависит не только от входа в данный момент, но и от того, что было раньше. Например, интегратор накапливает входной сигнал, а апериодическое звено плавно догоняет вход. Величины, которые звено помнит, называют переменными состояния и обозначают x.



Рисунок 1 – Звено: вход u, состояние x, выход y

Модель в пространстве состояний — это два уравнения:

$$\dot{x} = A\,x + B\,u,\; y = C\,x + D\,u. \qquad (1)$$

Первое уравнение говорит, как быстро меняется состояние ($\dot{x}$ — производная по времени). Второе — как из состояния и входа получается выход. Матрицы — это просто таблицы коэффициентов:

– A показывает, как состояние влияет само на себя (например, затухание);

– B — как вход влияет на изменение состояния;

– C — какая комбинация переменных состояния видна на выходе;

– D — какая часть входа проходит на выход сразу, без задержки.

Если такая модель построена для всей схемы, из неё без дополнительных преобразований получаются основные характеристики системы [1, 2]:

– полюса системы — это собственные числа матрицы A, то есть корни уравнения $det(sI - A) = 0$; система устойчива, если у всех полюсов вещественная часть отрицательна;

– передаточная функция — $W(s)=C(sI-A)^{-1}B+D$;

– переходный процесс — решение первого уравнения при ступенчатом входе;

– частотная характеристика — $W(j\omega)$.

Поэтому модель всей схемы в пространстве состояний — основа для всех расчётов системы моделирования.

1.2 Модели типовых звеньев

Для каждого звена матрицы A_i, B_i, C_i, D_i выписываются по его параметрам один раз. Покажем это на апериодическом звене $W(s)=\frac{K}{Ts+1}$. Его уравнение во времени: $T\dot{y}+y=Ku$. Возьмём в качестве состояния сам выход, $x=y$. Тогда

$$\dot{x}=-\frac{1}{T}x+\frac{K}{T}u,\ y=x, \tag{2}$$

то есть $A=-1/T$, $B=K/T$, $C=1$, $D=0$. У интегратора $\frac{k}{s}$ состояние — накопленное значение: $A=0$, $B=k$, $C=1$, $D=0$. У усилителя и сумматора состояния нет совсем, выход появляется сразу: для усилителя $D=k$, для сумматора D — строка из $+1$ и -1 по знакам входов. Звено с произвольной передаточной функцией $\frac{N(s)}{D(s)}$ записывается в стандартной (канонической) форме [2]: порядок модели равен степени знаменателя. Все поддерживаемые звенья приведены в таблице 1.

Таблица 1 – Типовые звенья и их модели

Звено	**Передаточная функция**	**Состояний**	**Мгновенная передача**
Усилитель	k	0	да
Сумматор	$\pm u_1 \pm u_2 \pm \dots$	0	да
Интегратор	$\frac{k}{s}$	1	нет
Звено 1-го порядка	$\frac{K}{T s + 1}$	1	нет
Колебательное звено	$\frac{K \omega_0^2}{s^2 + 2\zeta \omega_0 s + \omega_0^2}$	2	нет
Передаточная функция	$\frac{N(s)}{D(s)}$	степень D(s)	если степени N и D равны
ФНЧ Баттерворта	порядок n, частота среза ω_c	n	нет
ПИД-регулятор	$K_p + \frac{K_i}{s} + \frac{K_d N s}{s + N}$	2	да

Последний столбец показывает, передаёт ли звено изменение входа на выход сразу ($D_i \neq 0$). Это свойство понадобится в разделе 2.6: если сигнал по петле проходит только через такие звенья, он обходит её мгновенно, и такую петлю приходится решать отдельно.

1.3 Как получить модель всей схемы

Можно выделить три способа (таблица 2).

Таблица 2 – Способы получить модель всей схемы

Способ	**Как работает**	**Ограничения**
Правила структурных преобразований (вручную)	Последовательные звенья перемножают, параллельные складывают, контур сворачивают по формуле $W/(1+W)$	Долго и легко ошибиться, если контуров несколько; даёт только передаточную функцию
Поблочный расчёт (как при обычном моделировании в Simulink)	На каждом шаге времени блоки считаются по очереди, по ходу сигнала	Нет общих матриц системы; не работает, если сигнал обходит петлю мгновенно
Построение единой модели (разработано в проекте)	Уравнения всех звеньев объединяются, соединения подставляются, получаются A, B, C, D	Только линейные звенья

Поблочный расчёт использовался в первой версии системы моделирования. Он прост, но не даёт матриц всей системы и не может

рассчитать схему, в которой сигнал проходит по петле без задержки. Поэтому в проекте разработан третий способ. Он не ищет на схеме контуры и не выбирает порядок их сворачивания: любое соединение для него — просто ещё одна запись в таблице соединений.

2 Алгоритм построения модели

2.1 Общий порядок работы

Алгоритм состоит из шести шагов (рисунок 2). Если на каком-либо шаге модель построить нельзя, пользователь получает сообщение с причиной и названиями блоков, которые нужно исправить.

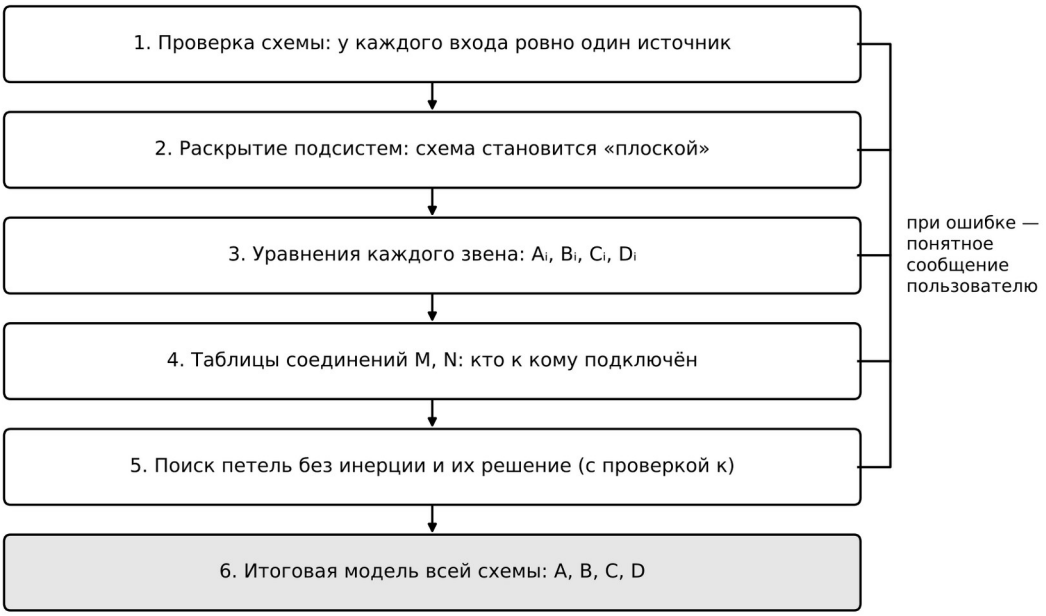


Рисунок 2 – Порядок построения модели по структурной схеме

2.2 Проверка схемы

Схема приходит на сервер в виде списка блоков и списка соединений «выход блока → вход блока» (формат JSON, пример — в разделе 4.2). Сначала проверяется, что схему вообще можно рассчитать: все блоки известного типа, параметры допустимы (например, постоянная времени $T>0$), к каждому входу подключён ровно один сигнал, нет соединений с несуществующими портами. Ошибки собираются списком, чтобы пользователь увидел их все сразу.

2.3 Раскрытие подсистем

Подсистема — это блок, внутри которого своя схема со своими входами и выходами. Для расчёта она не нужна как отдельный объект,

поэтому алгоритм сначала «раскрывает» подсистемы: переносит их блоки на верхний уровень и соединяет связи, проходившие через границу подсистемы, напрямую. Вложенные подсистемы раскрываются так же, рекурсивно.

Чтобы имена блоков из разных подсистем не совпали, к имени каждого блока добавляется путь, например measurement::sensor_path::filter. Поэтому две копии одной подсистемы не влияют друг на друга, а пользователь потом видит, из какой подсистемы пришла каждая переменная модели. Если выход подсистемы соединён прямо с её входом, сигнал просто проходит насквозь.

2.4 Объединение уравнений звеньев

После раскрытия у каждого звена i есть свои матрицы A_i, B_i, C_i, D_i (раздел 1.2). Запишем уравнения всех звеньев вместе, не учитывая пока соединений. Состояния всех звеньев собираются в один вектор x, все входы — в вектор u_b, все выходы — в вектор y_b. Матрицы звеньев ставятся по диагонали общей матрицы, остальное заполняется нулями:

$$A_d = \begin{pmatrix} A_1 & 0 & \cdots \\ 0 & A_2 & \cdots \\ \vdots & \vdots & \ddots \end{pmatrix}, \dot{x} = A_d x + B_d u_b,\ y_b = C_d x + D_d u_b. \tag{3}$$

Матрицы B_d, C_d, D_d составляются так же. Пока это просто «звенья, лежащие рядом».

2.5 Таблицы соединений

Теперь нужно учесть, как звенья соединены. Каждый вход получает ровно один сигнал: либо выход другого блока, либо внешнее воздействие r (ступенчатый вход). Это записывается двумя таблицами из нулей и единиц:

$$u_b = M\, y_b + N\, r. \tag{4}$$

В таблице M строки — входы блоков, столбцы — выходы блоков. Единица в строке «вход усилителя» и столбце «выход сумматора» означает, что выход сумматора подключён ко входу усилителя. Таблица N так же показывает, к каким входам подключены внешние воздействия. Аналогично

записывается, что видят осциллографы: $y = S_y y_b + S_r r$. Таблицы составляются автоматически, проходом по списку соединений схемы.

2.6 Итоговые матрицы и петли без инерционных звеньев

Подставим соединения $u_b = M y_b + N r$ в уравнение выходов звеньев. Выходы оказываются связаны между собой:

$$\left(I - D_d M\right) y_b = C_d x + D_d N r, \tag{5}$$

где I — единичная матрица. Это обычная система линейных уравнений относительно y_b. Решив её, получаем выходы всех блоков через состояние и воздействия: $y_b = Y_x x + Y_r r$. Подставив их в уравнения состояния и осциллографов, получаем модель всей схемы:

$$A = A_d + B_d M Y_x, B = B_d\left(M Y_r + N\right), C = S_y Y_x, D = S_y Y_r + S_r. \tag{6}$$

Решать всю систему уравнений целиком обычно не нужно. В большинстве схем сигнал идёт от блока к блоку, и выходы можно найти по очереди, подставляя уже найденные. Поэтому алгоритм сначала находит группы блоков, по которым сигнал может пройти по кругу мгновенно (через звенья с $D_i \neq 0$). Для этого используется поиск сильно связных компонент графа — алгоритм Тарьяна [5]. Остальные блоки считаются простой подстановкой по ходу сигнала, а систему уравнений приходится решать только внутри найденных групп.

Такая группа называется алгебраической петлёй. Её решение может быть очень чувствительным к коэффициентам. Простейший пример — усилитель k с положительной обратной связью:

$$y = k\left(u + y\right) \Rightarrow y = \frac{k}{1 - k} u. \tag{7}$$

При $k = 1$ решения нет. При k, близком к единице, решение есть, но малая неточность в k сильно меняет результат: при $k = 0{,}999$ получаем $y = 999\,u$, а при $k = 0{,}998$ — уже $y = 499\,u$. Чувствительность петли оценивается числом обусловленности κ: оно показывает, во сколько раз относительная погрешность коэффициентов может вырасти в ответе [4]. Для петли из

нескольких блоков G — матрица мгновенных передач внутри петли, σ_{min} — наименьшее сингулярное число:

$$\kappa = \frac{1 + \| G \|}{\sigma_{min}(I - G)}, \text{ для одного усилителя } \kappa = \frac{1 + |k|}{|1 - k|}. \quad (8)$$

Компьютер хранит числа примерно с 16 верными знаками. Если κ больше $1/\sqrt{\varepsilon} \approx 6,7 \cdot 10^{7}$, где $\varepsilon \approx 2,2 \cdot 10^{-16}$ — машинная точность, в ответе остаётся меньше половины верных знаков. В этом случае подсистема не выдаёт сомнительный результат, а сообщает, какие блоки образуют петлю, и предлагает изменить коэффициенты или добавить инерционное звено.

3 Примеры построения модели

Разберём работу алгоритма на шести схемах. Для каждой модель сначала строится вручную, по шагам главы 2, а затем сравнивается с тем, что выдала программа. Ответы программы получены сценарием backend/scripts/kp_assembly_study.py и показаны во вкладке «Матрицы» интерфейса.

3.1 Последовательное соединение

Схема показана на рисунке 3: ступенчатое воздействие r проходит через апериодическое звено $\frac{2}{0,5s+1}$ и интегратор $\frac{1}{s}$.

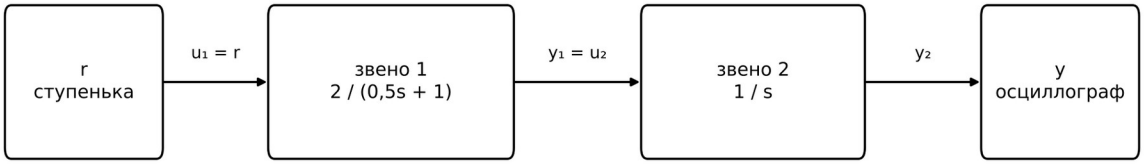


Рисунок 3 – Последовательное соединение двух звеньев

Шаг 1. Уравнения звеньев. По разделу 1.2 ($K=2$, $T=0,5$):

$$\dot{x}_1=-2x_1+4u_1,\ y_1=x_1;\ \dot{x}_2=u_2,\ y_2=x_2. \tag{9}$$

Шаг 2. Объединение. Матрицы звеньев ставятся по диагонали:

$$A_d=\begin{pmatrix}-2 & 0\\ 0 & 0\end{pmatrix},\ B_d=\begin{pmatrix}4 & 0\\ 0 & 1\end{pmatrix},\ C_d=I,\ D_d=0. \tag{10}$$

Шаг 3. Таблицы соединений. Вход первого звена подключён к воздействию ($u_1=r$), вход второго — к выходу первого ($u_2=y_1$), осциллограф показывает y_2:

$$M=\begin{pmatrix}0 & 0\\ 1 & 0\end{pmatrix},\ N=\begin{pmatrix}1\\ 0\end{pmatrix},\ S_y=\begin{pmatrix}0 & 1\end{pmatrix}. \tag{11}$$

Шаг 4. Итоговые матрицы. Мгновенной передачи у звеньев нет ($D_d=0$), поэтому петель без инерции нет и $y_b=x$, то есть $Y_x=I$, $Y_r=0$. По формулам раздела 2.6:

$$A = A_d + B_d M = \begin{pmatrix} -2 & 0 \\ 1 & 0 \end{pmatrix}, B = B_d N = \begin{pmatrix} 4 \\ 0 \end{pmatrix}, C = \begin{pmatrix} 0 & 1 \end{pmatrix}, D = 0. \quad (12)$$

Первая строка A и B — это уравнение первого звена, вторая строка — уравнение интегратора, вход которого теперь равен x_1. Проверим передаточную функцию: $C(sI - A)^{-1}B = \frac{4}{s(s+2)}$, что совпадает с ручным расчётом $\frac{2}{0,5s+1} \cdot \frac{1}{s}$. Программа выдала те же матрицы (рисунок 4); расхождение передаточных функций — $1{,}1 \cdot 10^{-16}$, то есть ошибка округления.

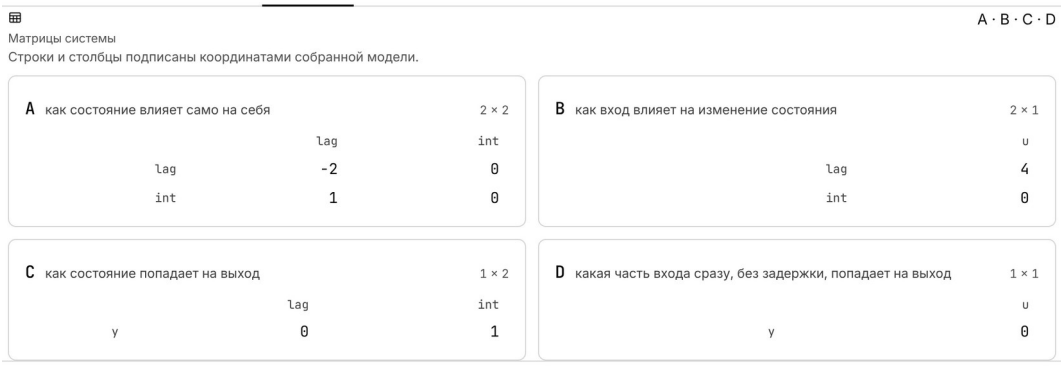


Рисунок 4 – Матрицы последовательного соединения во вкладке «Матрицы»

3.2 Параллельное соединение

Здесь (рисунок 5) сигнал разветвляется: одна ветвь идёт через звено $\frac{1}{s+1}$, другая — через усилитель $k = 2$, затем ветви складываются.

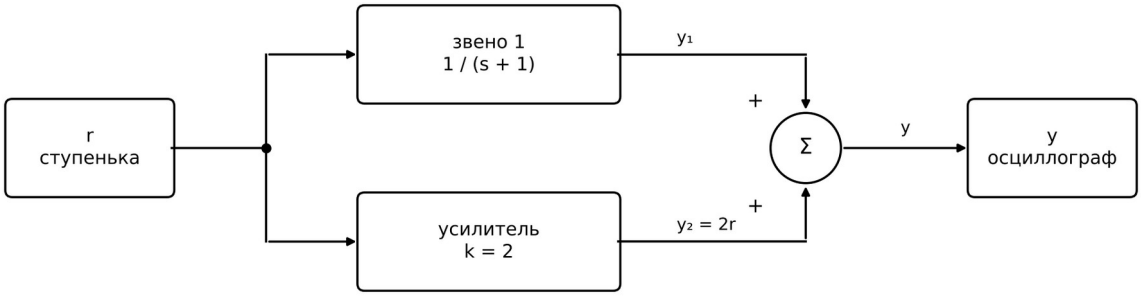


Рисунок 5 – Параллельное соединение

Состояние одно — у апериодического звена: $\dot{x} = -x + r$, $y_1 = x$. Усилитель и сумматор состояния не имеют и передают сигнал сразу. Программа

находит выходы по ходу сигнала: сначала $y_1 = x$ и $y_2 = 2r$, затем выход сумматора $y = y_1 + y_2 = x + 2r$. Отсюда

$$A = -1, B = 1, C = 1, D = 2. \quad (13)$$

Здесь впервые $D \neq 0$: часть воздействия (через усилитель) попадает на выход мгновенно. Передаточная функция $\frac{1}{s+1} + 2 = \frac{2s+3}{s+1}$ совпадает с ручной. На рисунке 6 отмечена матрица D (1), найденная программой.



Рисунок 6 – Матрицы параллельного соединения: (1) — прямая передача D = 2

3.3 Замкнутый контур с неустойчивым объектом

На рисунке 7 неустойчивый объект $\frac{1}{s-1}$ охвачен усилителем $K = 3$ и отрицательной обратной связью.

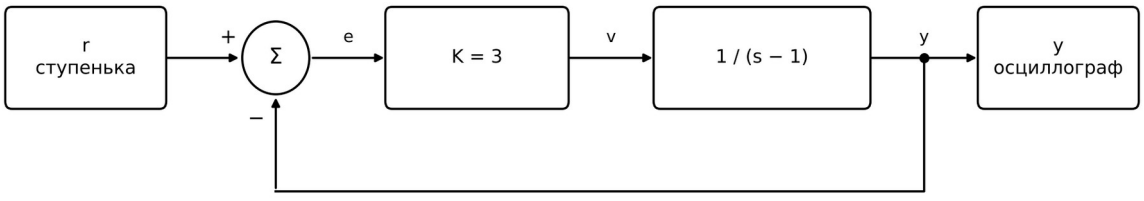


Рисунок 7 – Стабилизация неустойчивого объекта

У схемы три блока с выходами (сумматор, усилитель, объект) и четыре входа. Таблица соединений M приведена в таблице 3: например, единица в строке «усилитель» и столбце «выход сумматора» означает, что выход сумматора подключён ко входу усилителя. Вход «+» сумматора подключён к воздействию, поэтому он записан не в M, а в N.

Таблица 3 – Таблица соединений M для схемы рисунка 7

Вход блока	**Выход сумматора**	**Выход усилителя**	**Выход объекта**
сумматор, вход «+» (от ступеньки)	0	0	0
сумматор, вход «−»	0	0	1
усилитель	1	0	0
объект $\frac{1}{s-1}$	0	1	0

Объект: $\dot{x}=x+u_{об}$, $y_{об}=x$. Решая уравнения выходов по ходу сигнала, получаем $e=r-x$, $v=3e=3r-3x$. Подставляем v на вход объекта: $\dot{x}=x+3r-3x=-2x+3r$. Значит, $A=-2$, $B=3$, $C=1$, $D=0$, что совпадает с ручным расчётом по формуле контура: $W=\frac{3}{s-1+3}=\frac{3}{s+2}$. Обратите внимание: хотя у схемы есть контур, петли без инерции здесь нет — сигнал по контуру проходит через объект, у которого $D=0$. Полюс замкнутой системы — собственное число A — равен -2: замкнутая система устойчива, хотя объект сам по себе неустойчив. Ответ программы — на рисунке 8.

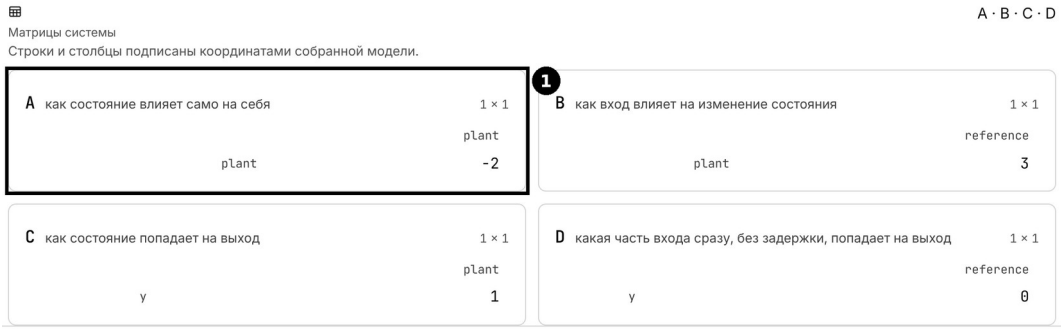


Рисунок 8 – Матрицы контура: (1) — матрица A = −2, полюс замкнутой системы

3.4 Тот же контур внутри подсистемы

Поместим усилитель и объект в подсистему «object» (рисунок 9). Для пользователя это другая схема: на верхнем уровне три блока, а внутри подсистемы свои входной и выходной порты.

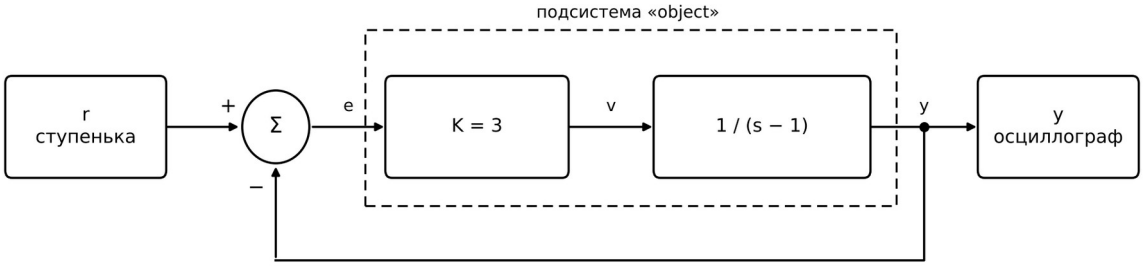


Рисунок 9 – Контур, в котором усилитель и объект вынесены в подсистему

При раскрытии (раздел 2.3) блоки подсистемы получают имена object::k и object::plant, а связи «сумматор → вход подсистемы» и «выход подсистемы → сумматор» заменяются прямыми связями с этими блоками. После этого схема совпадает со схемой раздела 3.3, и программа выдаёт те же матрицы: $A=-2$, $B=3$. Отличается только имя переменной состояния — object::plant: по нему видно, из какой подсистемы она пришла.

3.5 Контур третьего порядка

Схема на рисунке 10: объект $\frac{1}{s(s+1)(s+2)}$ с усилителем $K=2$ в контуре отрицательной обратной связи. Это пример, где модель объекта уже имеет три переменные состояния.

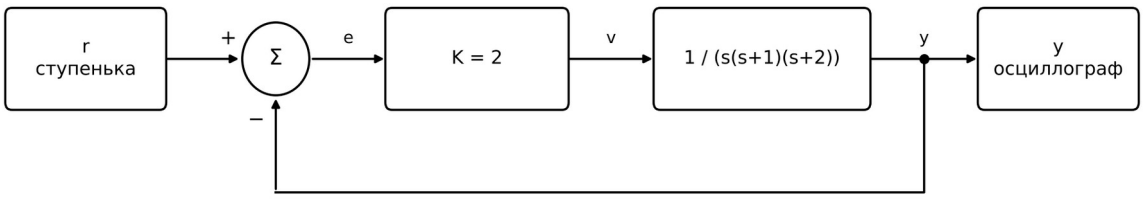


Рисунок 10 – Контур третьего порядка

Знаменатель объекта s^3+3s^2+2s. В канонической форме [2] модель объекта имеет матрицы

$$A_{\text{об}}=\begin{pmatrix}-3 & -2 & 0\\ 1 & 0 & 0\\ 0 & 1 & 0\end{pmatrix},\ B_{\text{об}}=\begin{pmatrix}1\\0\\0\end{pmatrix},\ C_{\text{об}}=\begin{pmatrix}0 & 0 & 1\end{pmatrix}. \qquad (14)$$

Обратная связь подаёт на вход объекта $2(r - x_3)$. Это добавляет в первую строку матрицы A слагаемое $-2x_3$:

$$A=\begin{pmatrix} -3 & -2 & -2 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{pmatrix}, B=\begin{pmatrix} 2 \\ 0 \\ 0 \end{pmatrix}, C=\begin{pmatrix} 0 & 0 & 1 \end{pmatrix}, D=0. \quad (15)$$

Матрица A из программы (рисунок 11, отметка 1) совпадает с ручной. Её характеристический многочлен $det(sI - A)=s^3+3s^2+2s+2$ — это знаменатель передаточной функции замкнутой системы $\frac{2}{s^3+3s^2+2s+2}$, полученной по правилу контура. Корни многочлена: $-2{,}52$ и $-0{,}24 \pm 0{,}86j$ — все левее мнимой оси, значит, система устойчива. Расхождение передаточных функций — $8{,}5 \cdot 10^{-17}$.

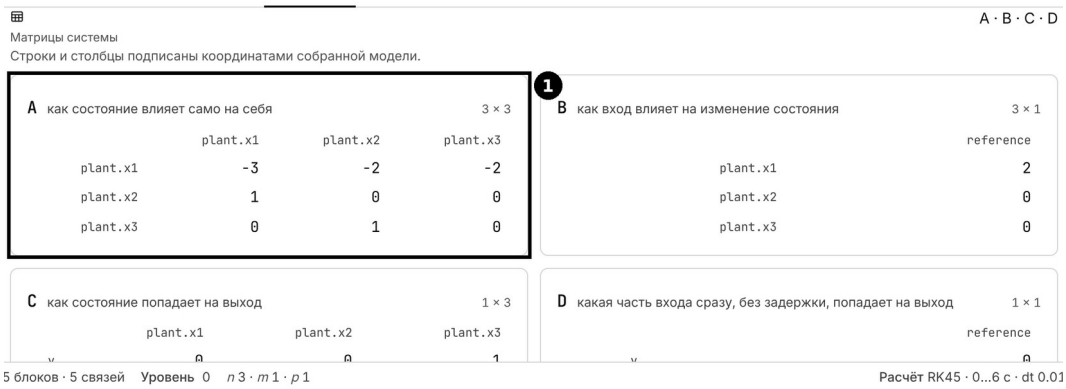


Рисунок 11 – Матрицы контура третьего порядка: (1) — матрица A 3×3

3.6 Петля без инерционных звеньев

На рисунке 12 усилитель k охвачен положительной обратной связью, и на пути сигнала по кругу нет ни одного инерционного звена. Здесь у модели нет состояний совсем, а выход связан со входом алгебраическим уравнением.



Рисунок 12 – Петля без инерционных звеньев

Выходы сумматора и усилителя зависят друг от друга: $e = r + y$, $y = k e$. Программа находит эту пару блоков как алгебраическую петлю и решает систему из двух уравнений. При $k = 0,5$ получаем $y = \frac{0,5}{1-0,5} r = r$, то есть $D = 1$ — как и выдала программа. Число обусловленности петли к = 7,12: петля надёжна. Оно больше, чем по формуле для одного усилителя ($\frac{1+0,5}{0,5} = 3$), потому что петля проходит через два блока. Сведения о петле выводятся во вкладке «Матрицы» (рисунок 13).

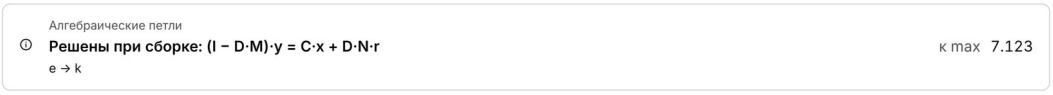


Рисунок 13 – Сведения о решённой петле без инерционных звеньев

При $k = 1$ уравнение $y = y + r$ решения не имеет. Программа не строит модель и сообщает причину с названиями блоков петли (рисунок 14, отметка 1).

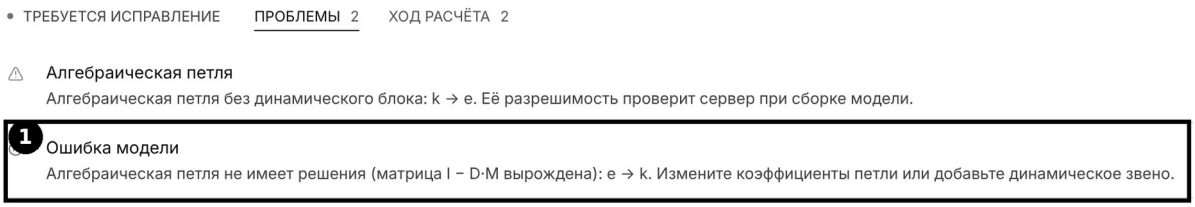


Рисунок 14 – Сообщение о петле, не имеющей решения: (1) — причина и блоки петли

Результаты всех примеров сведены в таблицу 4.

Таблица 4 – Примеры: ручной расчёт и программа

Пример	**Передаточная функция (вручную)**	**Расхождение**
последовательное соединение	W = 2/(0,5s+1) · 1/s = 4/(s(s+2))	$1,1 \cdot 10^{-16}$
параллельное соединение	W = 1/(s+1) + 2 = (2s+3)/(s+1)	$1,8 \cdot 10^{-16}$
обратная связь	W = 3/(s−1) / (1 + 3/(s−1)) = 3/(s+2)	$1,1 \cdot 10^{-16}$
обратная связь через подсистему	та же схема, звенья внутри подсистемы: W = 3/(s+2)	$1,1 \cdot 10^{-16}$
контур 3-го порядка	W = 2/(s³+3s²+2s) / (1 + 2/(s³+3s²+2s)) = 2/(s³+3s²+2s+2)	$8,5 \cdot 10^{-17}$

4 Программная реализация

4.1 Место подсистемы в системе моделирования

Система моделирования состоит из веб-интерфейса (React, TypeScript) и сервера (Python, FastAPI). Пользователь собирает схему в браузере и нажимает «Запустить». Интерфейс отправляет схему на сервер, сервер строит модель, рассчитывает по ней графики и характеристики и возвращает результат. Разработанная подсистема — первый шаг этого расчёта: все остальные модули сервера работают уже с матрицами A, B, C, D. Файлы подсистемы перечислены в таблице 5.

Таблица 5 – Файлы подсистемы

Файл	Назначение
backend/app/validation/validator.py	проверка схемы (раздел 2.2)
backend/app/simulation/hierarchy.py	раскрытие подсистем (раздел 2.3)
backend/app/simulation/assembly.py	модели звеньев, таблицы соединений, петли без инерции, итоговые матрицы (разделы 2.4–2.6)
backend/app/simulation/model.py	общий порядок: проверка → раскрытие → построение модели
backend/app/analysis/system.py	полюса и передача модели в ответ сервера
frontend/src/components/simulation/ StateSpacePanel.tsx	вкладка «Матрицы» в интерфейсе
backend/scripts/kp_assembly_study.py	примеры и проверка (главы 3 и 5)

4.2 Входные данные

Схема передаётся на сервер в формате JSON: список блоков с типом и параметрами и список соединений. Каждое соединение указывает блок и порт, откуда идёт сигнал (from), и блок и порт, куда он приходит (to). Ниже — схема примера 3.3 (координаты блоков на холсте опущены):

```
{
  "blocks": [
    {"id": "k", "type": "Gain", "parameters": {"k": 3.0}},
    {"id": "plant", "type": "TransferFunction",
     "parameters": {"numerator": [1.0], "denominator": [1.0, -1.0]}},
    {"id": "u", "type": "StepInput",
     "parameters": {"amplitude": 1.0, "t0": 0.0}},
    {"id": "e", "type": "Sum", "parameters": {"signs": ["+", "-"]}},
    {"id": "y", "type": "Scope", "parameters": {"label": "y"}}
  ],
  "connections": [
    {"from_block": "k", "from_port": "out",
     "to_block": "plant", "to_port": "in"},
    {"from_block": "u", "from_port": "out",
```

```
     "to_block": "e", "to_port": "in1"},
    {"from_block": "e", "from_port": "out",
     "to_block": "k", "to_port": "in"},
    {"from_block": "plant", "from_port": "out",
     "to_block": "y", "to_port": "in"},
    {"from_block": "plant", "from_port": "out",
     "to_block": "e", "to_port": "in2"}
  ]
}
```

4.3 Основные структуры данных

Основные структуры данных подсистемы приведены в таблице 6.

Таблица 6 – Структуры данных

Структура	Что хранит
BlockRealization	матрицы одного звена A_i, B_i, C_i, D_i и начальное состояние
StateEntry	какому блоку принадлежит переменная состояния и её номер внутри блока
AlgebraicLoop	блоки петли без инерции и её число обусловленности *к*
LinearModel	итоговые матрицы A, B, C, D, начальное состояние, список воздействий и осциллографов, найденные петли
ModelAssemblyError	список сообщений, если модель построить нельзя

4.4 Функции построения модели

Модель звена по его типу и параметрам строит функция block_realization (листинг 3 приложения А). Главная функция assemble_linear_model (листинг 1) проходит по блокам раскрытой схемы, объединяет их матрицы, заполняет таблицы соединений по списку связей и вычисляет итоговые матрицы по формулам раздела 2.6. Решение уравнений для выходов по группам блоков и проверку петель выполняет функция _solve_algebraic_equations (листинг 2), а порядок групп определяет функция _topological_components (листинг 4). Раскрытие подсистем выполняет функция _expand_level (листинг 5).

Для вычислений используются библиотеки NumPy и SciPy [6, 7]: из SciPy взяты поиск сильно связных компонент графа, LU-разложение для решения петель и перевод передаточной функции в каноническую форму.

4.5 Что получает пользователь

Построенная модель возвращается в ответе сервера вместе с результатами моделирования. В интерфейсе её показывает вкладка «Матрицы» (рисунок 15). В ней выводятся:

– матрицы A, B, C, D, у которых строки и столбцы подписаны именами блоков;

– таблица «Как собрана модель»: какому блоку и какой подсистеме принадлежит каждая переменная состояния;

– найденные петли без инерционных звеньев и их число обусловленности κ;

– размеры модели в строке состояния: n — число состояний, m — входов, p — выходов.

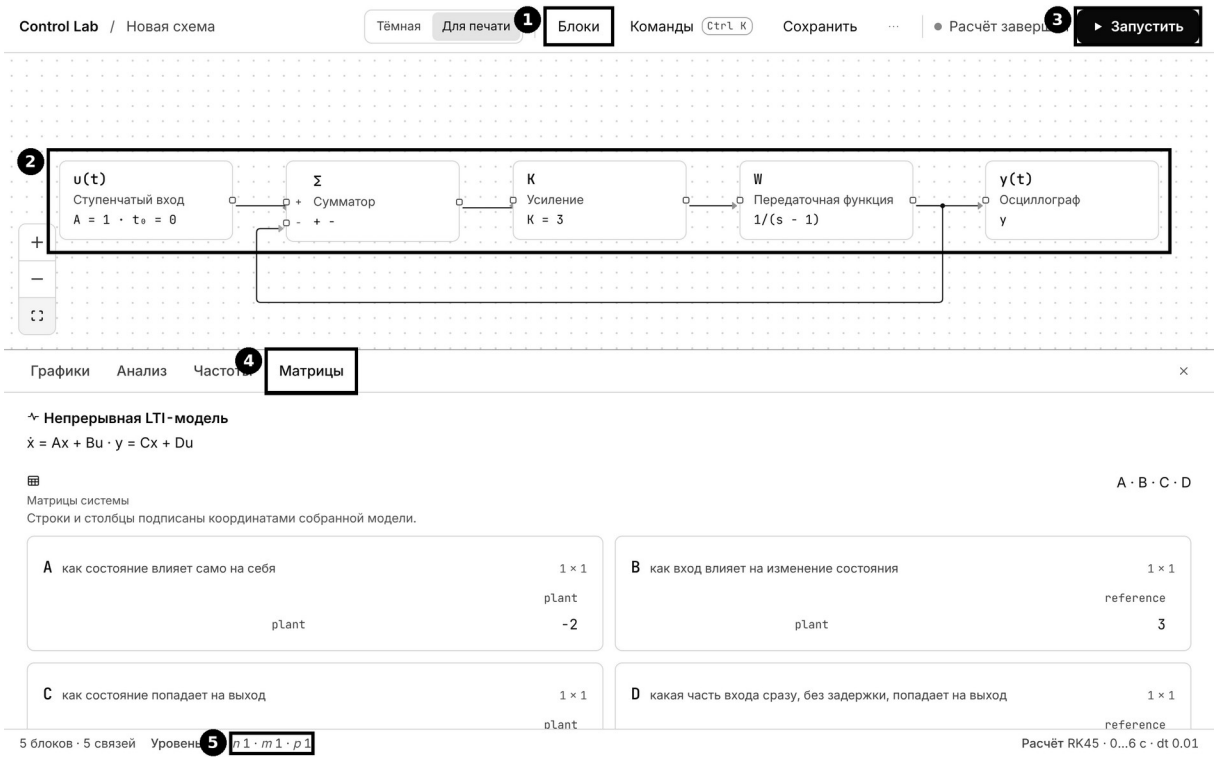


Рисунок 15 – Окно системы: (1) библиотека блоков, (2) схема, (3) запуск расчёта, (4) вкладка «Матрицы», (5) размеры модели

5 Проверка подсистемы

Проверка выполнена сценарием, который строит все таблицы и графики этой главы:

```
cd backend
python scripts/kp_assembly_study.py
```

5.1 Схемы с известным ответом

Для проверки подготовлено 16 схем: последовательное и параллельное соединение, разветвление сигнала, отрицательная и положительная обратная связь, контур с ПИД-регулятором, петли без инерционных звеньев, подсистемы, две копии одной подсистемы, подсистема с двумя входами и выходами, фильтр Баттерворта. Для каждой схемы передаточная функция $G(s)$ найдена вручную по правилам структурных преобразований.

Передаточная функция построенной модели $C(sI-A)^{-1}B+D$ сравнивалась с ручной в четырёх точках s. Ошибка считалась как $\frac{max\left|C(sI-A)^{-1}B+D-G(s)\right)}{max\left(1,\left|G(s)\right|\right)}$. Результаты приведены в таблице 7; последний столбец — результат поблочного расчёта первой версии системы.

Таблица 7 – Сравнение построенных моделей с ручным расчётом

Схема	n	Входы × выходы	Петли без инерции	Ошибка	Поблочный расчёт
последовательное соединение	2	1 × 1	0	$1{,}8\cdot10^{-16}$	$1{,}8\cdot10^{-16}$
параллельное соединение	2	1 × 1	0	$1{,}6\cdot10^{-16}$	$1{,}6\cdot10^{-16}$
разветвление	1	1 × 2	0	0	0
отрицательная обратная связь	2	1 × 1	0	$4{,}7\cdot10^{-16}$	$4{,}7\cdot10^{-16}$
положительная обратная связь	1	1 × 1	0	0	0
контур с ПИД-регулятором	3	1 × 1	0	$6{,}1\cdot10^{-16}$	$6{,}1\cdot10^{-16}$
петля через звено с D ≠ 0	1	1 × 1	1	$1{,}1\cdot10^{-16}$	не собирает
петля из усилителя	0	1 × 1	1	0	не собирает
контур через подсистему	1	1 × 1	0	$1{,}1\cdot10^{-16}$	$1{,}1\cdot10^{-16}$
тот же контур без подсистемы	1	1 × 1	0	$1{,}1\cdot10^{-16}$	$1{,}1\cdot10^{-16}$
две копии подсистемы	2	1 × 1	0	$5{,}6\cdot10^{-17}$	$5{,}6\cdot10^{-17}$
две копии с внутренними осциллографами	2	1 × 3	0	$5{,}6\cdot10^{-17}$	$5{,}6\cdot10^{-17}$
сквозной проход через подсистему	1	1 × 1	0	0	0

вложенные контуры	1	1 × 1	0	$1{,}1\cdot10^{-16}$	$1{,}1\cdot10^{-16}$
подсистема 2 входа × 2 выхода	1	2 × 2	0	0	0
фильтр Баттерворта 4-го порядка	4	1 × 1	0	$8{,}9\cdot10^{-16}$	$8{,}9\cdot10^{-16}$

Подсистема построила модели всех 16 схем. Наибольшая ошибка — $8{,}9\cdot10^{-16}$: это уровень погрешности округления чисел в компьютере, то есть результат совпадает с ручным расчётом. Поблочный расчёт не справился с 2 схемами — обе содержат петлю без инерционных звеньев.

5.2 Надёжность решения петель без инерционных звеньев

Исследовалась петля $y = k(u + y)$ при приближении k к единице. Для каждого k ответ программы сравнивался с точным $y = \frac{k}{1-k}$. Кроме того, k изменялся на одну миллиардную долю и измерялось, во сколько раз это изменение выросло в ответе. Данные — в таблице 8 и на рисунке 16.

Таблица 8 – Решение петли y = k(u + y)

k	**к (формула)**	**к (программа)**	**y программы**	**y точное**	**Ошибка**	**Рост погрешности**	**Итог**
0,5	3	7,12	1	1	0	2	решён
0,9	19	39	9	9	0	10	решён
0,99	199	399	99	99	0	100	решён
0,999	2e+03	4e+03	999	999	0	1e+03	решён
0,999999	2e+06	4e+06	999999	999999	0	1e+06	решён
0,999999999999	2e+12	—	—	1e+12	—	—	отказ: плохо обусловлен
1	∞	—	—	∞	—	—	отказ: вырожден
1,5	5	11,4	-3	-3	$1{,}5\cdot10^{-16}$	2	решён
3	2	6,83	-1,5	-1,5	$1{,}5\cdot10^{-16}$	0,5	решён

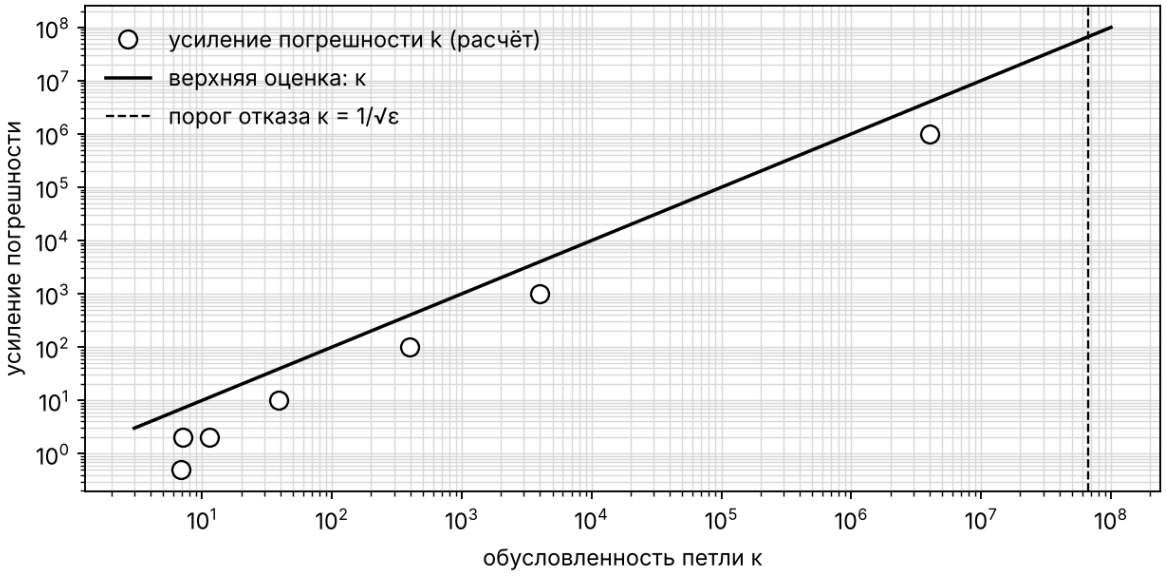


Рисунок 16 – Рост погрешности в зависимости от обусловленности петли

Пока κ невелико, программа решает петлю точно. Чем ближе k к единице, тем больше κ и тем сильнее растёт погрешность. Все точки на рисунке лежат ниже прямой κ, значит, оценка надёжна. При $k = 1 - 10^{-12}$ ($\kappa \approx 2 \cdot 10^{12}$) программа отказывается от расчёта, а при $k = 1$ сообщает, что петля не имеет решения.

5.3 Схемы с подсистемами

Сначала сравнивались две записи одной системы: контур, проходящий через подсистему, и та же схема без подсистем. Выходы совпали полностью (расхождение 0). Затем звено 1-го порядка в контуре с обратной связью вкладывалось в подсистемы глубиной от 1 до 32 уровней (таблица 9).

Таблица 9 – Раскрытие вложенных подсистем

Глубина	Блоков после раскрытия	n	Раскрытие, мс	Построение, мс	Отличие y(t)
1	4	1	0,11	0,60	0
2	4	1	0,11	0,71	0
4	4	1	0,14	0,74	0
8	4	1	0,17	0,86	0
16	4	1	0,38	1,25	0
32	4	1	1,14	1,99	0

При любой глубине после раскрытия остаются те же четыре блока, модель первого порядка и тот же выход. Время раскрытия растёт

пропорционально глубине и даже при 32 уровнях составляет доли миллисекунды. Как подсистемы отражаются в модели многоуровневой схемы, показано в таблице 10: для каждой переменной указаны блок и путь подсистем.

Таблица 10 – Переменные состояния многоуровневой схемы

x	Блок	Тип звена	Подсистема
x1	actuator_lag	звено 1-го порядка	plant / actuator
x2	oscillator	колебательное	plant
x3	oscillator	колебательное	plant

5.4 Время построения модели

Время измерялось на цепочке звеньев 1-го порядка длиной от 2 до 256 звеньев: без обратных связей и с обратной связью вокруг каждого звена. Каждый замер повторялся 15 раз, в таблицу 11 и на рисунок 17 занесено среднее значение (медиана). Время включает проверку схемы и раскрытие подсистем.

Таблица 11 – Время построения модели

n	Блоков	Время, мс	Блоков (с контурами)	Время, мс (с контурами)
2	4	0,46	6	0,64
8	10	0,93	18	1,41
32	34	2,09	66	5,60
64	66	3,99	130	14,13
128	130	13,63	258	21,71
256	258	35,20	514	76,58

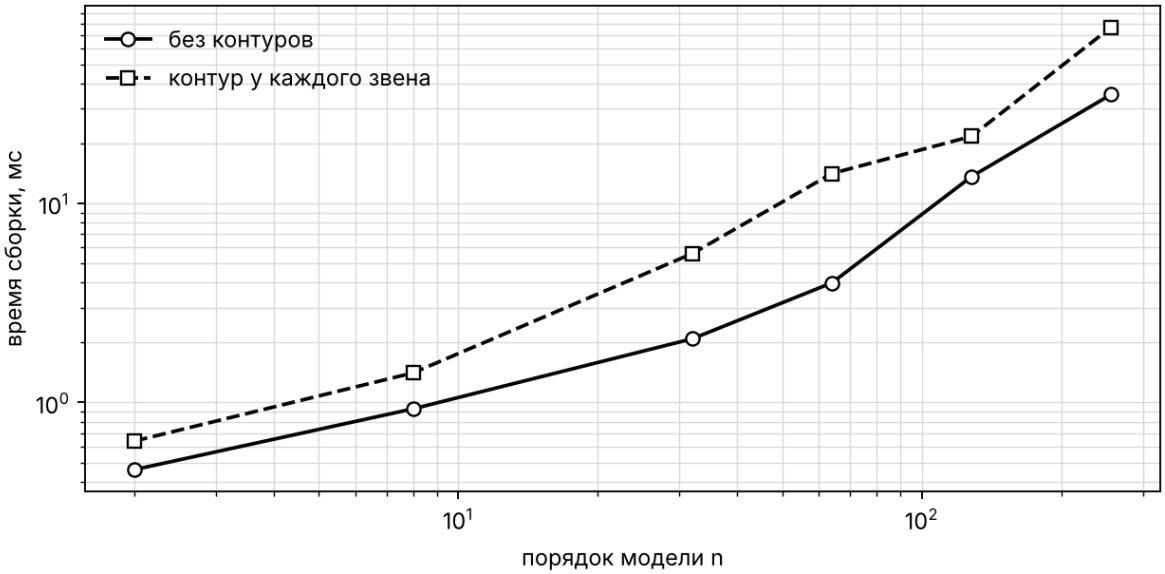


Рисунок 17 – Время построения модели в зависимости от её порядка

Даже схема из более чем 500 блоков с 256 контурами обрабатывается быстрее чем за десятую долю секунды, а учебные схемы из десятков блоков — за доли миллисекунды. Поэтому модель можно перестраивать при каждом изменении параметра, и пользователь сразу видит результат.

6 Руководство пользователя

6.1 Построение модели схемы

Основные элементы окна отмечены цифрами на рисунке 15 (раздел 4.5).

1. Откройте систему моделирования в браузере. Выберите готовый пример на пустом холсте или соберите схему из блоков: кнопка «Блоки» (1) открывает библиотеку, блоки перетаскиваются на холст (2), выход блока соединяется со входом другого блока мышью.

2. Нажмите «Запустить» (3) или Ctrl+Enter.

3. Откройте вкладку «Матрицы» (4). В ней — матрицы A, B, C, D и таблица «Как собрана модель». Размеры модели выводятся в строке состояния (5).

4. Чтобы изменить параметр, щёлкните блок и введите новое значение в поле (1) на рисунке 18. Модель будет перестроена при следующем запуске.

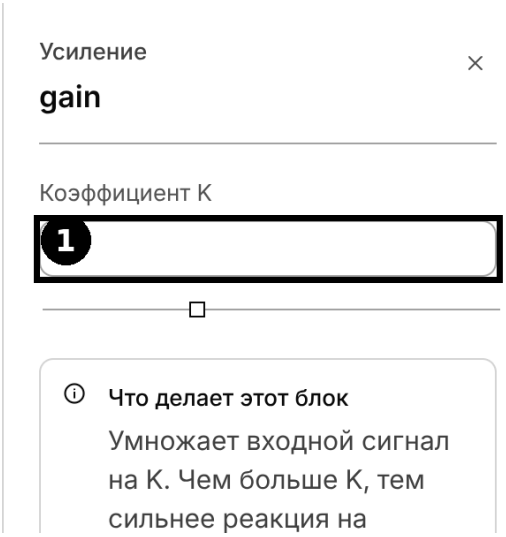


Рисунок 18 – Панель параметров блока: (1) — поле коэффициента

6.2 Работа с подсистемами

Подсистема на схеме отмечена надписью SUB (рисунок 19, отметка 1). Двойной щелчок открывает её содержимое (рисунок 20). После запуска таблица «Как собрана модель» показывает путь подсистемы для каждой переменной (рисунок 21).

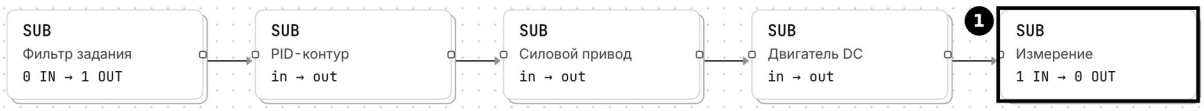


Рисунок 19 – Схема из подсистем: (1) — подсистема «Измерение»

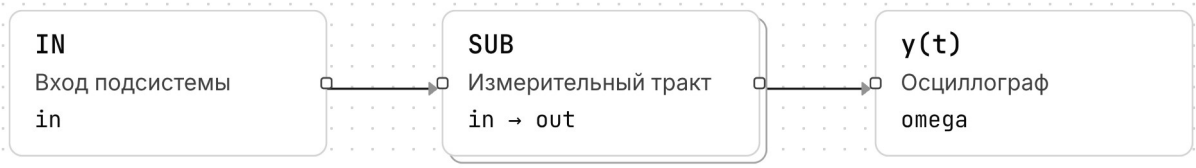


Рисунок 20 – Содержимое подсистемы «Измерение»

Как собрана модель

Координаты x связаны с динамическими блоками исходной схемы.

Координата	Состояние	Блок	Подсистема
x_1	**controller::pid.x1**	controller::pidPIDController	controller
x_2	**controller::pid.x2**	controller::pidPIDController	controller
x_3	**drive::drive_lag**	drive::drive_lagFirstOrderLag	drive
x_4	**motor::motor_model.x1**	motor::motor_modelSecondOrderOscillator	motor
x_5	**motor::motor_model.x2**	motor::motor_modelSecondOrderOscillator	motor
x_6	**measurement::sensor_path::adc_stage::signal_filter::filter**	measurement::sensor_path::adc_stage::signal_filter::filterFirstOrderLag	measurement / s

Рисунок 21 – Таблица «Как собрана модель» для схемы с подсистемами

6.3 Если модель не построена

Если модель построить нельзя, вместо результатов открывается панель «Проблемы» (рисунок 14). Каждое сообщение называет причину и блоки, которые нужно исправить. Чаще всего это неподключённый вход, недопустимый параметр или петля без инерционных звеньев. В последнем случае помогает изменить коэффициент петли или добавить в неё инерционное звено.

Заключение

В ходе курсового проекта разработана подсистема построения математической модели САУ в пространстве состояний по структурной схеме для веб-ориентированной системы моделирования.

Разработан алгоритм, который по схеме любой сложности строит матрицы A, B, C, D всей системы. Схема проверяется, подсистемы раскрываются, уравнения звеньев объединяются, соединения записываются таблицами, а уравнения выходов решаются по ходу сигнала. Петли без инерционных звеньев решаются отдельно с оценкой надёжности; ненадёжные и нерешаемые петли не дают сомнительного результата, а сопровождаются понятным сообщением.

Подсистема реализована в серверной части, а полученная модель показывается во вкладке «Матрицы»: пользователь видит матрицы и то, какому блоку и какой подсистеме принадлежит каждая переменная.

Работа алгоритма разобрана по шагам на шести примерах. Проверка на 16 схемах с известным ответом показала совпадение с ручным расчётом с точностью до погрешности округления ($8{,}9 \cdot 10^{-16}$), тогда как поблочный расчёт не справился с 2 из них. Глубина вложенности подсистем не влияет на результат, а модель из более чем 500 блоков строится за десятки миллисекунд.

Разработанная подсистема является основой выпускной квалификационной работы: по построенной модели выполняются моделирование, анализ устойчивости и частотный анализ.

Список литературы

1. Бесекерский В.А., Попов Е.П. Теория систем автоматического регулирования. – М.: Директмедиа, 2021. – 747 с.

2. Ким Д.П. Теория автоматического управления. Т. 1. Линейные системы. – М.: Физматлит, 2016. – 312 с.

3. Ogata K. Modern Control Engineering. 5th ed. – Upper Saddle River: Prentice Hall, 2010. – 894 p.

4. Голуб Дж., Ван Лоун Ч. Матричные вычисления. – М.: Мир, 1999. – 548 с.

5. Кормен Т., Лейзерсон Ч., Ривест Р., Штайн К. Алгоритмы: построение и анализ. 3-е изд. – М.: Вильямс, 2013. – 1328 с.

6. NumPy Documentation [Электронный ресурс]. – URL: https://numpy.org/doc/ (дата обращения: 01.10.2026).

7. SciPy API Reference [Электронный ресурс]. – URL: https://docs.scipy.org/doc/scipy/reference/ (дата обращения: 01.10.2026).

Приложение А. Листинги программы

Листинг 1 – Построение модели схемы (app/simulation/assembly.py)

```
def assemble_linear_model(flat: Diagram) -> LinearModel:
    """Assemble a validated, flattened diagram into (A, B, C, D)."""

    sources: list[StepSource] = []
    scopes: list[ScopeOutput] = []
    source_index: dict[str, int] = {}
    scope_index: dict[str, int] = {}

    dynamic: list[tuple[Block, BlockRealization]] = []
    output_index: dict[str, int] = {}      # block id -> row of y_b
    input_offset: dict[str, int] = {}      # block id -> first row of u_b
    states: list[StateEntry] = []
    total_inputs = 0

    for block in flat.blocks:
        if block.type in SOURCE_TYPES:
            source_index[block.id] = len(sources)
            sources.append(
                StepSource(
                    block.id,
                    get_numeric_parameter(block.parameters, "amplitude", 1.0),
                    get_numeric_parameter(block.parameters, "t0", 0.0),
                )
            )
            continue
        if block.type in SINK_TYPES:
            label = str(block.parameters.get("label") or "").strip() or block.id
            reference = (
                get_numeric_parameter(block.parameters, "reference", 0.0)
                if "reference" in block.parameters
                else None
            )
            scope_index[block.id] = len(scopes)
            scopes.append(ScopeOutput(block.id, label, reference))
            continue

        realization = block_realization(block)
        if realization.b.shape[1] != len(block.input_ports):
            raise ModelAssemblyError(
                [f"Блок '{block.id}': число входов не совпадает с моделью звена."]
            )
        output_index[block.id] = len(dynamic)
        input_offset[block.id] = total_inputs
        total_inputs += len(block.input_ports)
        for local in range(realization.order):
            label = block.id if realization.order == 1 else f"{block.id}.x{local + 1}"
            states.append(StateEntry(block.id, block.type, local, label))
        dynamic.append((block, realization))

    realizations = [realization for _, realization in dynamic]
    n = sum(r.order for r in realizations)
    n_out = len(dynamic)
    n_src = len(sources)
    p = len(scopes)

    a_d = _block_diagonal([r.a for r in realizations], n, n)
    b_d = _block_diagonal([r.b for r in realizations], n, total_inputs)
    c_d = _block_diagonal([r.c for r in realizations], n_out, n)
    d_d = _block_diagonal([r.d for r in realizations], n_out, total_inputs)
    x0 = np.concatenate([r.x0 for r in realizations]) if realizations else np.zeros(0)

    m = np.zeros((total_inputs, n_out))
    nr = np.zeros((total_inputs, n_src))
    s_y = np.zeros((p, n_out))
    s_r = np.zeros((p, n_src))
```

```
    blocks_by_id = {block.id: block for block in flat.blocks}

    for connection in flat.connections:
        target = blocks_by_id[connection.to_block]
        if target.type in SINK_TYPES:
            row_matrix_y, row_matrix_r, row = s_y, s_r, scope_index[target.id]
        else:
            row = input_offset[target.id] +
target.input_ports.index(connection.to_port)
            row_matrix_y, row_matrix_r = m, nr
        if connection.from_block in source_index:
            row_matrix_r[row, source_index[connection.from_block]] = 1.0
        else:
            row_matrix_y[row, output_index[connection.from_block]] = 1.0

    g = d_d @ m
    owners = [block.id for block, _ in dynamic]
    y_solution, loops = _solve_algebraic_equations(g, np.hstack([c_d, d_d @ nr]),
owners)
    y_x, y_r = y_solution[:, :n], y_solution[:, n:]

    return LinearModel(
        a=a_d + b_d @ m @ y_x,
        b=b_d @ (m @ y_r + nr),
        c=s_y @ y_x,
        d=s_y @ y_r + s_r,
        x0=x0,
        states=states,
        sources=sources,
        scopes=scopes,
        algebraic_loops=loops,
    )
```

Листинг 2 – Решение уравнений выходов и петель без инерции (app/simulation/assembly.py)

```
def _solve_algebraic_equations(
    g: np.ndarray,
    rhs: np.ndarray,
    owners: list[str],
) -> tuple[np.ndarray, list[AlgebraicLoop]]:
    """Solve (I - g) Y = rhs component by component."""

    solution = np.zeros_like(rhs)
    loops: list[AlgebraicLoop] = []
    eps = np.finfo(float).eps
    for component in _topological_components(g):
        index = np.asarray(component)
        local_rhs = rhs[index] + g[index] @ solution  # unsolved rows of solution are
still zero
        if index.size == 1 and g[index[0], index[0]] == 0.0:
            solution[index] = local_rhs
            continue

        loop_blocks = sorted({owners[i] for i in component})
        loop_gain = g[np.ix_(index, index)]
        matrix = np.eye(index.size) - loop_gain
        smallest = float(np.linalg.svd(matrix, compute_uv=False)[-1])
        # Relative sensitivity of the loop solution to perturbations of the
        # loop coefficients: kappa = (1 + ||G||) / sigma_min(I - G). For a
        # scalar loop y = k y + ... this is (1 + |k|) / |1 - k|.
        scale = 1.0 + float(np.linalg.norm(loop_gain, 2))
        if smallest <= eps * scale * index.size:
            raise ModelAssemblyError(
                [
                    "Алгебраическая петля не имеет решения (матрица I − D·M
вырождена): "
                    + " → ".join(loop_blocks)
```

```
                    + ". Измените коэффициенты петли или добавьте динамическое звено."
                ]
            )
        condition = scale / smallest
        if condition > MAX_LOOP_CONDITION:
            raise ModelAssemblyError(
                [
                    f"Алгебраическая петля плохо обусловлена (κ = {condition:.3g}): "
                    + " → ".join(loop_blocks)
                    + ". Результат расчёта был бы недостоверным."
                ]
            )
        solution[index] = lu_solve(lu_factor(matrix), local_rhs)
        loops.append(AlgebraicLoop(blocks=loop_blocks, condition_number=condition))
    return solution, loops
```

Листинг 3 – Модели типовых звеньев (app/simulation/assembly.py)

```
def block_realization(block: Block) -> BlockRealization:
    p = block.parameters
    kind = block.type
    if kind == "Gain":
        return _static(np.array([[get_numeric_parameter(p, "k", 1.0)]]))
    if kind == "Sum":
        signs = get_signs(p)
        return _static(np.array([[1.0 if sign == "+" else -1.0 for sign in signs]]))
    if kind == "Integrator":
        k = get_numeric_parameter(p, "k", 1.0)
        return BlockRealization(
            a=np.zeros((1, 1)), b=np.array([[k]]), c=np.ones((1, 1)), d=np.zeros((1,
1)),
            x0=np.array([get_numeric_parameter(p, "y0", 0.0)]),
        )
    if kind == "FirstOrderLag":
        k = get_numeric_parameter(p, "k", 1.0)
        t_const = get_numeric_parameter(p, "T", 1.0)
        return BlockRealization(
            a=np.array([[-1.0 / t_const]]), b=np.array([[k / t_const]]),
            c=np.ones((1, 1)), d=np.zeros((1, 1)),
            x0=np.array([get_numeric_parameter(p, "y0", 0.0)]),
        )
    if kind == "SecondOrderOscillator":
        k = get_numeric_parameter(p, "k", 1.0)
        wn = get_numeric_parameter(p, "wn", 1.0)
        zeta = get_numeric_parameter(p, "zeta", 0.2)
        return BlockRealization(
            a=np.array([[0.0, 1.0], [-wn * wn, -2.0 * zeta * wn]]),
            b=np.array([[0.0], [k * wn * wn]]),
            c=np.array([[1.0, 0.0]]),
            d=np.zeros((1, 1)),
            x0=np.array([get_numeric_parameter(p, "y0", 0.0), get_numeric_parameter(p,
"v0", 0.0)]),
        )
    if kind == "TransferFunction":
        return _transfer_function_realization(*get_transfer_function_coefficients(p))
    if kind == "PIDController":
        return _transfer_function_realization(*get_pid_coefficients(p))
    if kind == "ButterworthLPF":
        order = int(get_numeric_parameter(p, "order", 2))
        realization = _transfer_function_realization(
            *butterworth_coefficients(order, get_numeric_parameter(p, "cutoff_freq",
10.0))
        )
        x0 = _minimum_norm_state(realization.c, get_numeric_parameter(p, "y0", 0.0))
        return BlockRealization(realization.a, realization.b, realization.c,
realization.d, x0)
    raise ValueError(f"Блок '{block.id}' типа '{kind}' не может входить в плоскую
модель.")
```

Листинг 4 – Порядок групп блоков (app/simulation/assembly.py)

```
def _topological_components(g: np.ndarray) -> list[list[int]]:
    """Strongly connected components of 'j feeds i' (g[i, j] != 0), sources first."""

    count = g.shape[0]
    if count == 0:
        return []
    adjacency = csr_matrix((g != 0.0).T.astype(np.int8))  # edge j -> i
    n_components, labels = connected_components(adjacency, directed=True,
connection="strong")
    members: list[list[int]] = [[] for _ in range(n_components)]
    for index, label in enumerate(labels):
        members[label].append(index)

    successors: list[set[int]] = [set() for _ in range(n_components)]
    indegree = [0] * n_components
    rows, cols = np.nonzero(g)
    for i, j in zip(rows, cols, strict=True):
        source, target = labels[j], labels[i]
        if source != target and target not in successors[source]:
            successors[source].add(target)
            indegree[target] += 1
    ready = sorted(c for c in range(n_components) if indegree[c] == 0)
    ordered: list[list[int]] = []
    while ready:
        component = ready.pop(0)
        ordered.append(members[component])
        for successor in sorted(successors[component]):
            indegree[successor] -= 1
            if indegree[successor] == 0:
                ready.append(successor)
    return ordered
```

Листинг 5 – Раскрытие подсистем (app/simulation/hierarchy.py)

```
def _expand_level(diagram: Diagram, prefix: str) -> DiagramExpansion:
    expansion = DiagramExpansion()
    blocks_by_id = {block.id: block for block in diagram.blocks}
    child_expansions: dict[str, DiagramExpansion] = {}
    incoming: dict[Endpoint, Endpoint] = {
        (connection.to_block, connection.to_port): (connection.from_block,
connection.from_port)
        for connection in diagram.connections
    }

    for block in diagram.blocks:
        if block.type == "Subsystem":
            child = _expand_level(_nested_diagram(block), f"{prefix}{block.id}
{PATH_SEPARATOR}")
            child_expansions[block.id] = child
            expansion.blocks.extend(child.blocks)
            expansion.connections.extend(child.connections)
        elif block.type not in {"SubsystemInput", "SubsystemOutput"}:
            expansion.blocks.append(_copy_block(block, prefix))

    def source_endpoint(block_id: str, port: str, visiting: frozenset[str] =
frozenset()) -> Endpoint:
        """Resolve the flattened signal source behind an output port of this level.

        Returns (INPUT_ALIAS, port) when the signal comes from an interface
        input of this level, so that the parent can resolve it further.
        """

        block = blocks_by_id[block_id]
        if block.type == "SubsystemInput":
            return (INPUT_ALIAS, _port_name(block))
        if block.type != "Subsystem":
            return (_prefix_id(prefix, block_id), port)
```

```
        source = child_expansions[block_id].output_sources.get(port)
        if source is None:
            raise ValueError(f"Подсистема '{block_id}' не формирует выходной порт
'{port}'.")
        if source[0] != INPUT_ALIAS:
            return source

        # The child passes one of its inputs straight through to this output.
        key = f"{block_id}.{source[1]}"
        if key in visiting:
            raise ValueError(f"Подсистема '{block_id}' замыкает собственный вход на
выход по кругу.")
        upstream = incoming.get((block_id, source[1]))
        if upstream is None:
            raise ValueError(f"Вход '{source[1]}' подсистемы '{block_id}' не
подключен.")
        return source_endpoint(upstream[0], upstream[1], visiting | {key})

    def target_endpoints(block_id: str, port: str) -> list[Endpoint] | None:
        block = blocks_by_id[block_id]
        if block.type == "SubsystemOutput":
            return None
        if block.type == "Subsystem":
            # An input that is only passed through has no internal targets; its
            # consumers are reached through the resolved output instead.
            return child_expansions[block_id].input_targets.get(port, [])
        return [(_prefix_id(prefix, block_id), port)]

    for connection in diagram.connections:
        target_block = blocks_by_id[connection.to_block]
        source = source_endpoint(connection.from_block, connection.from_port)

        if target_block.type == "SubsystemOutput":
            output_port = _port_name(target_block)
            if output_port in expansion.output_sources:
                raise ValueError(f"Выходной порт подсистемы '{output_port}' имеет
несколько источников.")
            expansion.output_sources[output_port] = source
            continue

        targets = target_endpoints(connection.to_block, connection.to_port) or []
        if source[0] == INPUT_ALIAS:
            expansion.input_targets.setdefault(source[1], []).extend(targets)
            continue
        for target in targets:
            expansion.connections.append(
                Connection(
                    from_block=source[0],
                    from_port=source[1],
                    to_block=target[0],
                    to_port=target[1],
                )
            )

    interface_inputs = [_port_name(block) for block in diagram.blocks if block.type ==
"SubsystemInput"]
    interface_outputs = [_port_name(block) for block in diagram.blocks if block.type
== "SubsystemOutput"]
    if len(set(interface_inputs)) != len(interface_inputs):
        raise ValueError("Имена входных портов подсистемы должны быть уникальными.")
    if len(set(interface_outputs)) != len(interface_outputs):
        raise ValueError("Имена выходных портов подсистемы должны быть уникальными.")
    passed_through = {source[1] for source in expansion.output_sources.values() if
source[0] == INPUT_ALIAS}
    for port in interface_inputs:
        if not expansion.input_targets.get(port) and port not in passed_through:
            raise ValueError(f"Входной порт подсистемы '{port}' не используется.")
    for port in interface_outputs:
        if port not in expansion.output_sources:
            raise ValueError(f"Выходной порт подсистемы '{port}' не подключен.")
```

```
return expansion
```